

MEMOIRE DE FIN DE FORMATION

REFERENTIEL

DEV DATA

INTITULE

Conception et réalisation d'une **Plateforme WOYOFAL** pour l'Analyse de
Données et Simulation de Recharges Électriques

Présenté et soutenu par :

Diarra NDOYE

Encadré par :

M. Mamadou MBAYE

Promo 7 Développement Data

Avant-propos

Dans un monde en pleine mutation numérique, les données constituent désormais le socle de toute prise de décision éclairée. La Sonatel Academy s'impose comme un acteur visionnaire au service de la jeunesse africaine, offrant un cadre d'apprentissage rigoureux et ancré dans les réalités du terrain.

Intégrée à l'écosystème dynamique de l'Orange Digital Center, cette école de code gratuite forme chaque année des talents en développement web, mobile, data et digital marketing. La pédagogie active, centrée sur la pratique et les projets concrets, permet aux apprenants de développer des compétences directement applicables.

Rejoindre la formation Développeur Data Cohorte 6 a représenté une opportunité unique de m'immerger dans un domaine en pleine expansion : celui de l'ingénierie des données. À l'ère du big data, les compétences en traitement, modélisation et visualisation de données sont devenues essentielles pour bâtir des solutions intelligentes et utiles.

C'est dans cet esprit que j'ai conçu le projet Woyofal, une plateforme d'analyse de données et de simulation de recharges électriques, avec l'ambition de contribuer, à mon échelle, à une meilleure compréhension de la consommation énergétique des ménages sénégalais.

Remerciements

Je tiens à exprimer ma profonde gratitude envers la Sonatel Academy pour l'opportunité de participer à cette formation spécialisée en développement data. Leur engagement en faveur de l'accessibilité des compétences numériques témoigne d'une volonté affirmée de préparer les jeunes talents du Sénégal aux métiers d'avenir.

Je remercie chaleureusement M. Mamadou MBAYE, notre formateur, dont l'expertise et la pédagogie ont été essentielles pour saisir les concepts techniques abordés tout au long de la formation. Son approche méthodique, mêlant théorie et application pratique, a permis d'aborder avec aisance des domaines variés : programmation, gestion de bases de données, analyse de données, pipelines ETL et bien plus encore.

Je tiens également à saluer mes camarades de la cohorte. L'entraide, les échanges et la collaboration quotidienne avec eux ont été une source de motivation et ont enrichi cette expérience d'apprentissage de manière significative.

Enfin, mes remerciements vont à toute l'équipe de la Sonatel Academy pour leur assistance logistique et technique, qui a grandement facilité un environnement d'apprentissage dynamique et propice à la réussite.

Table des matières

Avant-propos	2
Remerciements.....	3
Sommaire	Erreur ! Signet non défini.
Liste des figures	6
Liste des abréviations	7
Introduction	10
Présentation de l'entreprise et de son écosystème	11
1. Historique et mission de SONATEL	11
2. Rôle de l'Orange Digital Center (ODC) dans l'insertion professionnelle.....	11
3. Présentation de l'École du Code Sonatel Academy (ECSA).....	11
4. Impact de cet écosystème sur l'éducation numérique au Sénégal	11
Présentation du projet Woyofal	13
Problématique.....	13
Critique de l'existant.....	14
Objectifs du projet.....	19
Étapes de réalisation du projet	20
Architecture technique du projet.....	23
1. Architecture globale 7 couches	23
2. Architecture JAMstack Frontend React.....	23
3. Star Schema du Data Warehouse	24
4. Pipeline ETL Flux de traitement	24
Outils et technologies utilisées	26
1. Frontend : React 18 + Vite 5 + Tailwind CSS.....	26
2. Langage backend : Python 3.10+	26
3. Base de données : PostgreSQL 14.....	26
4. Génération de données : Faker + NumPy + SciPy	26
5. Interface analytique : Streamlit.....	26
6. Chatbot IA : OpenRouter API	27
7. Déploiement : Vercel + GitHub Actions	27
8. Tableau des bibliothèques complémentaires.....	27
Interfaces et visualisation.....	29
1. Application React Simulateur grand public	29
2. Moteur de calcul tarifaire Algorithmes clés	29
3. Dashboard Streamlit Espace analytique	30
4. Jupyter Notebooks Analyses EDA.....	30
Analyse technique du code.....	31
1. Structure générale du projet	31

2. Classe DataWarehouseLoader (ETL Python)	31
3. Classe RechargeSimulator (Python backend)	31
4. Chatbot IA Route /api/ai (Flask)	32
5. Sécurité et bonnes pratiques	32
6. Points forts de l'implémentation.....	32
Résultats et tests	34
1. Bilan des fonctionnalités livrées	34
2. Corrections apportées en cours de développement	34
Défis et leçons apprises	36
1. Défis rencontrés	36
2. Leçons apprises	36
Conclusion.....	38
Perspectives d'évolution.....	38
Bibliographie	40
Ouvrages imprimés	40
Cours en ligne et tutoriels.....	40
Webographie.....	41

Liste des figures

Figure 1	Architecture globale de la plateforme Woyofal (JAMstack)	11
Figure 2	Star Schema du Data Warehouse (PostgreSQL)	13
Figure 3	Pipeline ETL détaillé (Extract-Transform-Load)	15
Figure 4	Interface du simulateur de recharges (React mode direct)	20
Figure 5	Interface du simulateur de recharges (React mode inverse)	21
Figure 6	Dashboard Streamlit Analyse des consommations	22
Figure 7	Chatbot IA Assistant Woyofal (OpenRouter)	23
Figure 8	Graphiques EDA Distribution et tranches tarifaires	25

Liste des abréviations

Abréviation	Définition complète
DWH	Data Warehouse (Entrepôt de données)
ETL	Extract, Transform, Load
FCFA	Franc CFA (monnaie de l'Afrique de l'Ouest)
EDA	Exploratory Data Analysis (Analyse Exploratoire)
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
ECSA	École du Code Sonatel Academy
ODC	Orange Digital Center
SENELEC	Société Nationale d'Électricité du Sénégal
CRSE	Commission de Régulation du Secteur de l'Énergie
SQL	Structured Query Language
DPP	Domestique Petite Puissance (type compteur)
PPP	Professionnel Petite Puissance (type compteur)
SPA	Single Page Application
JWT	JSON Web Token
PWA	Progressive Web Application

Glossaire

Termes	Définitions
Data Warehouse	Base de données optimisée pour l'analyse et le reporting, organisant les données selon un schéma en étoile.
Star Schema	Modèle de conception de base de données composé d'une table de faits centrale liée à plusieurs tables de dimensions.
Pipeline ETL	Processus automatisé d'extraction, transformation et chargement des données depuis des sources vers une destination.
Woyofal	Terme wolof désignant la recharge d'électricité prépayée sur compteur électronique Senelec.
Tranche tarifaire	Palier de consommation électrique mensuelle associé à un prix unitaire spécifique (T1, T2, T3, T4).
Cumul mensuel	Total de kWh consommés depuis le début du mois, consultable sur le compteur via le code 814.
Redevance mensuelle	Frais fixe de 429 FCFA déduit lors de la première recharge du mois (location et entretien compteur).
Taxe communale	Prélèvement de 2,5 % appliqué sur chaque montant de recharge.
Streamlit	Framework Python open-source pour créer des applications web interactives de visualisation de données.
React.js	Bibliothèque JavaScript pour la construction d'interfaces utilisateur modernes et réactives.
PostgreSQL	Système de gestion de bases de données relationnelles open-source, utilisé comme Data Warehouse.
OpenRouter	API d'accès multi-modèles IA (Llama, Gemma, Mistral, Qwen) avec mécanisme de fallback automatique.
Vercel	Plateforme cloud de déploiement frontend avec CI/CD intégré et CDN mondial.

Termes	Définitions
JAMstack	Architecture web basée sur JavaScript, APIs et Markup calcul côté client, sans serveur requis.

Introduction

Dans un Sénégal en pleine transformation numérique, l'accès à l'électricité constitue un pilier fondamental du développement économique et social. Le système de recharges prépayées, communément appelé « Woyofal », est au cœur du quotidien de millions de ménages sénégalais. Le Sénégal compte plus de 17 millions d'habitants répartis dans 14 régions et 23 zones d'approvisionnement électrique gérées par la Senelec (Société Nationale d'Électricité du Sénégal).

Le système tarifaire sénégalais, régi par la Décision 2025-140 de la CRSE (Commission de Régulation du Secteur de l'Énergie), applique un barème progressif à plusieurs tranches selon la consommation mensuelle cumulée. Cette progressivité bien que socialement justifiée est d'une complexité opaque pour l'utilisateur final. Pourtant, la compréhension des tarifications, l'optimisation des recharges et l'analyse des comportements de consommation restent des défis peu adressés par les outils numériques existants.

C'est dans ce contexte que s'inscrit ce mémoire, fruit d'une formation rigoureuse au sein de la Sonatel Academy, dans le cadre du référentiel Développement Data. Le projet Woyofal est né d'une double volonté : d'une part, concevoir un simulateur fiable et accessible répondant aux besoins immédiats des consommateurs ; d'autre part, construire une plateforme de données complète permettant d'analyser, de visualiser et de valoriser les comportements de consommation électrique à l'échelle nationale.

En combinant des technologies modernes telles que React.js, Python, PostgreSQL et Streamlit, ce projet propose une solution technique robuste couvrant l'ensemble du cycle de vie des données : de la génération synthétique réaliste, à l'ingestion dans un Data Warehouse optimisé, jusqu'à la visualisation et la simulation interactive. La plateforme est déployée en production et accessible à l'adresse frontend-react-wine.vercel.app.

Ce mémoire a pour objectif de présenter la genèse, la réalisation et les perspectives de ce projet. Il met en lumière les motivations, les défis techniques, les choix architecturaux, mais aussi l'impact potentiel d'une telle plateforme au service des consommateurs sénégalais.

Présentation de l'entreprise et de son écosystème

L'environnement dans lequel s'est déroulée cette formation est un écosystème riche et innovant, impulsé par des acteurs majeurs du numérique au Sénégal.

1. Historique et mission de SONATEL

Créée en 1985, la Société Nationale des Télécommunications du Sénégal (SONATEL) est aujourd'hui un pilier des télécommunications en Afrique de l'Ouest. Elle s'est d'abord fixé pour mission de fournir des services de téléphonie sur l'ensemble du territoire sénégalais, avant d'élargir son périmètre pour devenir un acteur clé de la transformation digitale. En rejoignant le groupe Orange en 1997, Sonatel s'est inscrite dans une dynamique internationale tout en gardant une forte empreinte locale. Elle propose aujourd'hui une large gamme de services : téléphonie mobile, internet, services financiers numériques, cloud et solutions IT.

2. Rôle de l'Orange Digital Center (ODC) dans l'insertion professionnelle

L'Orange Digital Center (ODC) est une initiative stratégique du groupe Orange, présente dans plusieurs pays africains. Au Sénégal, l'ODC regroupe en un seul lieu plusieurs structures complémentaires : une école de code, un FabLab, un espace de coworking et un incubateur de start-ups. L'objectif principal est d'offrir aux jeunes talents un accès gratuit à des formations pratiques, des ressources techniques et un accompagnement entrepreneurial. Il joue un rôle central dans l'émergence d'un écosystème numérique solide, inclusif et durable.

3. Présentation de l'École du Code Sonatel Academy (ECSA)

La Sonatel Academy, également appelée École du Code, est un centre de formation gratuit en développement web, mobile, data et digital marketing. Créée par Sonatel en 2017, cette école vise à répondre aux besoins croissants en compétences numériques sur le marché sénégalais. Son modèle repose sur une pédagogie active, centrée sur la pratique, l'apprentissage par projet et la collaboration. La sélection se fait sur la motivation et non sur le diplôme.

4. Impact de cet écosystème sur l'éducation numérique au Sénégal

La synergie entre Sonatel, l'ODC et la Sonatel Academy produit un effet catalyseur sur le développement du numérique au Sénégal. Grâce à cet environnement, des milliers de jeunes peuvent aujourd'hui accéder à des formations de qualité, à des équipements modernes et à des réseaux professionnels. Ces initiatives participent activement à la réduction des inégalités d'accès à la technologie, à la création d'emplois durables, et à la structuration d'une économie numérique inclusive.

Présentation du projet Woyofal

Problématique

L'accès à une information tarifaire fiable et à des outils de simulation reste un défi majeur pour les consommateurs d'électricité prépayée au Sénégal. La grille tarifaire de SENELEC, complexifiée par quatre tranches progressives (T1, T2, T3, T4 pour les compteurs DPP) et des déductions variables (redevance, taxe communale), rend difficile pour le ménage moyen la compréhension de sa facture et l'optimisation de sa consommation.

Une enquête informelle auprès de ménages dakarois révèle que 87 % des utilisateurs ignorent leur cumul mensuel actuel avant de recharger, et que la quasi-totalité ne connaît pas le montant exact pour rester en tranche 1. Par ailleurs, bien que les données de consommation électrique représentent une mine d'informations précieuses pour la planification énergétique, elles restent largement sous-exploitées faute d'outils d'analyse adaptés.

Dans ce contexte, une question centrale se pose :

Comment concevoir une plateforme de données complète capable d'analyser les comportements de consommation électrique, de simuler les recharges Woyofal et de rendre ces informations accessibles et exploitables par tous ?

Ce projet s'inscrit dans un double défi :

- Un défi technique : comment construire un pipeline de données robuste, de la génération à la visualisation, en passant par un moteur de calcul tarifaire conforme à la Décision 2025-140 ?
- Un défi social : comment rendre compréhensible la tarification électrique pour aider les consommateurs à mieux gérer leur budget énergétique ?

Critique de l'existant

Avant de présenter la solution développée dans ce projet, il convient d'analyser les outils existants qui répondent partiellement à la problématique posée. Cette analyse comparative permet de justifier les choix de conception retenus et d'identifier les lacunes que la plateforme Woyofal vise à combler.

a. Présentation de la solution de référence : woyofal-check.site

Le site woyofal-check.site constitue la solution en ligne la plus avancée actuellement disponible pour les consommateurs Woyofal au Sénégal. Développé en 2025, il propose un calculateur de recharges, un suivi de consommation et des prédictions basiques. Son interface est accessible via navigateur web ou application PWA, sans installation requise.

Malgré son utilité certaine, une analyse approfondie révèle plusieurs limitations significatives, tant sur le plan fonctionnel que technique, qui justifient le développement d'une plateforme plus robuste et plus complète.

b. Limites fonctionnelles identifiées

b.1 Accès restreint aux fonctionnalités principales

Sur woyofal-check.site, un visiteur non connecté ne voit qu'une page de présentation et une FAQ. L'ensemble des fonctionnalités utiles (calculateur, suivi, prédictions) est conditionné à la création d'un compte. Cette friction à l'entrée exclut une partie des utilisateurs, notamment ceux peu à l'aise avec les processus d'inscription en ligne.

La plateforme Woyofal développée dans ce projet adopte une approche inverse : toutes les fonctionnalités de simulation et d'analyse sont accessibles sans inscription, à l'exception des fonctionnalités personnalisées (lecture de compteur et historique enregistré) qui nécessitent un compte.

b.2 Grille tarifaire non mise à jour

woyofal-check.site utilise une grille tarifaire antérieure à la Décision SENELEC 2025-140, avec des prix au kWh ne correspondant plus aux tarifs officiels en vigueur en 2026. Or, la précision du calcul tarifaire est le cœur de valeur de ce type d'outil : une erreur de grille entraîne des simulations inexactes, potentiellement sources de mauvaises décisions budgétaires pour les ménages.

La plateforme Woyofal implémente la grille officielle 2026 issue de la Décision 2025-140, avec une distinction rigoureuse entre les types DPP (T1 à 82 FCFA/kWh, T2/T3 à 136,49 FCFA/kWh, T4 à 162 FCFA/kWh) et PPP (T1 à 147,43 FCFA/kWh, T2/T3 à 189,84 FCFA/kWh).

b.3 Absence de prise en compte du cumul mensuel (code 814)

Le fonctionnement du système Woyofal est intrinsèquement mensuel : les tranches tarifaires se réinitialisent à chaque début de mois. Ainsi, pour toute recharge effectuée après la première du mois, le calcul des kWh obtenus dépend du cumul de consommation déjà enregistré sur le compteur consultable via le code 814.

woyofal-check.site ne prend pas en compte ce paramètre pour les recharges intermédiaires du mois, ce qui fausse mécaniquement les calculs. Un utilisateur ayant déjà consommé 120 kWh dans le mois obtiendra un résultat différent pour une recharge de 5 000 FCFA selon qu'il est encore en Tranche 1 ou qu'il bascule en Tranche 2. La plateforme Woyofal intègre ce paramètre comme entrée obligatoire pour toute simulation hors première recharge.

b.4 Pas d'enregistrement des simulations

Sur woyofal-check.site, il n'est pas possible de sauvegarder le résultat d'une simulation pour le consulter ultérieurement. Chaque calcul est ponctuel et éphémère. La plateforme Woyofal permet, après simulation, d'enregistrer le résultat dans un historique personnel (localStorage côté client, puis base de données une fois le backend déployé), facilitant ainsi le suivi mensuel.

b.5 Absence de chatbot et assistance limitée à une FAQ statique

Le seul support disponible sur woyofal-check.site est une FAQ statique, qui répond à un ensemble prédéfini de questions. Cette approche ne permet pas de répondre à des questions complexes ou contextuelles par exemple : « Si je recharge 8 000 FCFA sachant que j'ai déjà consommé 90 kWh ce mois, combien vais-je obtenir et dans quelle tranche ? »

La plateforme Woyofal intègre un chatbot IA (composant AIChat.jsx) basé sur l'API OpenRouter, avec une cascade de six modèles de langage (dont Llama 3.3 70B, Gemma 3, Qwen 3 et Mistral 7B) et un mécanisme de fallback local intelligent. Ce chatbot comprend les questions en langage naturel, effectue les calculs tarifaires à la volée et fournit des explications pédagogiques adaptées.

b.6 Bug de connexion et instabilité

Des bugs de connexion ont été constatés sur woyofal-check.site, rendant l'accès aux fonctionnalités connectées aléatoire. La plateforme Woyofal a fait l'objet d'une suite de tests de validation sur plusieurs cas réels et d'un pipeline CI/CD via GitHub Actions → Vercel, garantissant la stabilité des fonctionnalités à chaque mise à jour.

c. Limites techniques et architecturales

Au-delà des fonctionnalités visibles, woyofal-check.site présente des lacunes profondes sur le plan de l'architecture de données :

- Absence de Data Warehouse : aucun entrepôt de données structuré ne permet d'agréger, d'historiser et d'analyser les comportements de consommation à l'échelle nationale ou régionale.
- Absence de pipeline ETL : les données ne sont pas collectées, transformées et chargées de manière automatisée et traçable.
- Aucune visualisation analytique : il n'existe pas de dashboard permettant de visualiser les tendances de consommation ou les disparités entre zones géographiques.
- Pas d'API REST documentée : la solution ne peut pas être intégrée ou réutilisée par d'autres systèmes.

La plateforme Woyofal répond à chacune de ces lacunes avec une architecture en sept couches couvrant la génération de données synthétiques, l'ingestion ETL, le Data Warehouse PostgreSQL en Star Schema, les calculs avancés, les interfaces analytiques et le CI/CD.

d. Tableau comparatif synthétique

Critère	Plateforme Woyofal (ce projet)	woyofal-check.site (existant)	Impact
Calculateur FCFA → kWh	☑ Oui	☑ Oui	
Calcul inverse kWh → FCFA	☑ Oui (sans connexion)	⚠ Partiel (connecté)	Accessibilité accrue

Critère	Plateforme Woyofal (ce projet)	woyofal-check.site (existant)	Impact
Grille tarifaire 2026 (Décision 2025-140)	☑ Intégrée (4 tranches DPP)	✗ Tarifs antérieurs	Risque d'erreurs
Prise en compte cumul mensuel (code 814)	☑ Paramètre obligatoire	✗ Non vérifiée	Calculs faussés
Enregistrement des simulations	☑ Historique localStorage + BD	✗ Non	Suivi impossible
Chatbot IA intégré	☑ OpenRouter multi-modèles	✗ FAQ statique	Aide dynamique vs statique
Data Warehouse / pipeline ETL	☑ Star Schema 330 000 lignes	✗ Absent	Analyse macro impossible
Dashboard analytique (23 zones)	☑ Streamlit + Folium	✗ Absent	Pas de visualisation géo
API REST exploitable	☑ Flask /api/*	✗ Absent	Non réutilisable
Déploiement cloud CI/CD	☑ Vercel + GitHub Actions	⚠ PWA basique	Fiabilité différente
Stabilité / bugs connus	☑ Testé, corrigé	✗ Bug de connexion	Expérience dégradée

e. Positionnement de la solution développée

Cette analyse comparative met en évidence que la plateforme Woyofal ne se positionne pas comme une simple alternative, mais comme une solution plus ambitieuse, articulée autour de trois axes différenciants :

1. **la grille 2026 officielle (Décision 2025-140) est intégrée avec 4 tranches DPP et prise en compte du cumul mensuel pour des simulations précises.**Fiabilité tarifaire :
2. **un Data Warehouse structuré, un pipeline ETL automatisé et des dashboards interactifs permettent une analyse à l'échelle des 23 zones géographiques du Sénégal.**Profondeur analytique :
3. **un chatbot IA conversationnel rend la plateforme accessible à tous les profils d'utilisateurs, bien au-delà des FAQ statiques.**Intelligence artificielle et accessibilité :

Objectifs du projet

Objectif général

Le projet Woyofal vise à concevoir et implémenter une plateforme d'analyse de données complète, intégrant un pipeline ETL robuste, un Data Warehouse structuré en schéma en étoile, un simulateur de recharges conforme à la grille tarifaire SENELEC 2026, et des interfaces de visualisation interactives accessibles à tous. L'objectif principal est de valoriser les données de consommation électrique pour permettre une meilleure compréhension des comportements tarifaires et faciliter la prise de décision des consommateurs.

Objectifs spécifiques

- Générer et structurer un jeu de données réaliste Produire 330 000+ lignes de données synthétiques (consommation quotidienne, recharges, profils utilisateurs, 23 zones géographiques) représentatives des réalités du terrain sénégalais.
- Mettre en place un pipeline ETL robuste Développer un processus automatisé d'extraction, de transformation et de chargement des données vers un Data Warehouse PostgreSQL optimisé.
- Concevoir un Data Warehouse en Star Schema Implémenter un schéma en étoile conforme à la méthodologie Kimball, avec 4 dimensions et 2 tables de faits, pour des performances analytiques optimales.
- Développer un simulateur de recharges fiable Implémenter un moteur de calcul conforme à la Décision SENELEC 2025-140, gérant les tranches T1/T2/T3/T4 pour les types DPP et PPP, avec support des modes direct et inverse.
- Créer des interfaces de visualisation interactives Développer un dashboard Streamlit pour l'exploration des données et une application React.js moderne pour le simulateur grand public, déployée sur Vercel.
- Intégrer un assistant IA conversationnel Déployer un chatbot basé sur OpenRouter capable de répondre en langage naturel aux questions tarifaires et d'effectuer des calculs à la volée.
- Garantir la qualité avec des tests et CI/CD Valider le moteur de calcul sur des cas de test réels et automatiser le déploiement via GitHub Actions.

Étapes de réalisation du projet

Le projet Woyofal s'est construit de manière itérative, en suivant une démarche centrée sur les données et les utilisateurs finaux, combinant analyse du besoin, choix technologiques, prototypage rapide, tests en conditions réelles et amélioration continue.

1. Identification du besoin

La première étape a consisté à analyser les besoins des consommateurs d'électricité prépayée au Sénégal. Plusieurs constats clés ont été mis en évidence :

- La grille tarifaire SENELEC est complexe et peu accessible au grand public, avec 4 tranches progressives pour les compteurs DPP.
- Il n'existe pas d'outil simple permettant de simuler une recharge en temps réel avec les tarifs 2026 officiels.
- Le cumul mensuel (code 814) est ignoré par les outils existants, faussant les calculs hors première recharge du mois.
- Les données de consommation sont dispersées et inexploitées pour l'analyse à l'échelle régionale.
- 87 % des utilisateurs ignorent leur cumul mensuel avant de recharger (enquête informelle, Dakar 2026).

2. Sélection des outils et technologies adaptés

- Python 3.10+ : langage principal pour le pipeline de données, les scripts ETL et le simulateur backend.
- React 18 + Vite 5 + Tailwind CSS : frontend moderne avec moteur de calcul entièrement côté client (JAMstack), déployé sur Vercel.
- PostgreSQL 14 : Data Warehouse avec Star Schema pour le stockage et l'analyse optimisée.
- Streamlit : dashboard analytique pour l'exploration interactive des données.
- OpenRouter API : chatbot IA multi-modèles avec fallback automatique (Llama 3.3 70B, Gemma 3, Qwen 3, Mistral 7B).
- Docker Compose : containerisation pour un déploiement reproductible et portable.
- GitHub Actions : CI/CD pour le déploiement automatique sur Vercel à chaque push.

3. Génération des données synthétiques

En l'absence de données réelles disponibles, une phase de génération synthétique a été menée pour produire un jeu de données représentatif :

- 23 zones géographiques couvrant les 14 régions du Sénégal.
- 10 000 utilisateurs avec des profils réalistes (type compteur DPP/PPP, zone, consommation typique).
- 300 000 lignes de consommation quotidienne sur 12 mois avec variations saisonnières.
- 20 000 transactions de recharges avec montants et fréquences réalistes.

4. Phase de tests et ajustements

Des tests approfondis ont été menés sur le moteur de calcul tarifaire en validant les résultats sur des cas connus :

Scénario	Résultat (kWh)	Tranche finale	Statut
5 000 FCFA, cumul=0, sans redevance	60,9 kWh	T1	☑ Validé
10 000 FCFA, cumul=0, sans redevance	118,9 kWh	T1	☑ Validé
15 000 FCFA, cumul=0, sans redevance	150 (T1) + 10,1 (T2) = 160,1 kWh	T2	☑ Validé
5 000 FCFA, cumul=0, avec redevance	55,7 kWh	T1	☑ Validé
Inverse : 150 kWh, cumul=0	12 300 FCFA	T1 (seuil exact)	☑ Validé
Inverse : 100 kWh, cumul=50	8 200 FCFA	T2	☑ Validé

Ces retours ont permis d'apporter plusieurs corrections importantes, notamment le déplacement du moteur de calcul entièrement côté client (résolution d'un bug de page

blanche en production lié aux appels à localhost:5000), la correction d'une référence d'état React obsolète et la suppression d'une variable d'environnement Vercel inexistante.

5. Valorisation de l'approche data-driven

L'un des grands principes du projet est la valeur par les données. Le système a été conçu pour :

- Produire des insights exploitables sur les comportements de consommation par zone et par tranche.
- Permettre des simulations précises pour aider les décisions budgétaires des ménages.
- Mettre en évidence les inégalités tarifaires entre zones géographiques grâce à la carte Folium.
- Offrir une accessibilité totale sans serveur requis pour les fonctionnalités de base (architecture JAMstack).

Architecture technique du projet

L'architecture du projet Woyofal repose sur une approche duale : d'une part, une architecture JAMstack côté frontend garantissant la disponibilité mobile sans serveur ; d'autre part, une architecture data en sept couches côté backend pour le traitement et l'analyse des données.

1. Architecture globale 7 couches

Couche	Description
1 Sources	Fichiers CSV : zones_senegal.csv, users.csv, consumption_daily.csv, recharges.csv
2 Ingestion	Validation (nulls, plages, intégrité), retry automatiques, logging complet
3 Transformation	Calcul tranches T1/T2/T3/T4, économies, agrégations par zone et par mois
4 Data Warehouse	Star Schema Kimball : 4 dimensions + 2 tables de faits (PostgreSQL)
5 Calculs avancés	Simulateur de recharges, prédictions ML, rapports agrégés
6 Analytics	Streamlit dashboards, Jupyter Notebooks EDA, Plotly, Folium
7 CI/CD	GitHub Actions, pytest, Docker Compose, déploiement Vercel automatique

Le frontend React est déployé indépendamment sur Vercel avec un moteur de calcul JavaScript pur, entièrement exécuté dans le navigateur (offline-first). Cette séparation garantit la disponibilité de la simulation même sans backend opérationnel, ce qui a permis de résoudre le problème critique de page blanche sur mobile rencontré en début de projet.

2. Architecture JAMstack Frontend React

L'application React adopte une architecture JAMstack (JavaScript, API, Markup) pour maximiser la disponibilité mobile et minimiser les coûts d'infrastructure. Le moteur de calcul tarifaire est dupliqué en JavaScript pur côté client, garantissant une disponibilité hors connexion pour les fonctionnalités de simulation.

L'identité visuelle repose sur trois couleurs : Rouge #DC2626 (boutons, accents), Marine #1A1D2E (fonds, navigation), Blanc #FFFFFF (fond général). La centralisation des couleurs dans `tailwind.config.js` a permis de maintenir la cohérence entre les 10+ composants React.

3. Star Schema du Data Warehouse

Le Data Warehouse suit le modèle en étoile (Star Schema) de la méthodologie Kimball, organisé autour de 4 tables de dimensions et 2 tables de faits :

Table	Colonnes principales	Type
dim_date	date_id (PK), date, jour, mois, trimestre, annee	Dimension
dim_zones	zone_id (PK), zone, region, ville, population	Dimension
dim_users	user_id (PK), prenom, nom, type_compteur, zone_id (FK)	Dimension
dim_tranches	tranche_id (PK), nom, seuil_min, seuil_max, prix_kwh, type_compteur	Dimension
fact_consumption	date_id (FK), user_id (FK), zone_id (FK), tranche_id (FK), conso_kwh, montant_fcfa	Fait (300k lignes)
fact_recharges	date_id (FK), user_id (FK), montant_brut, kwh_obtenus, tranche_avant, tranche_apres	Fait (20k lignes)

4. Pipeline ETL Flux de traitement

- **EXTRACT** : Lecture des fichiers CSV depuis le répertoire `data/raw/`.
- **VALIDATE** : Vérification des nulls, des plages de valeurs et de l'intégrité référentielle.
- **TRANSFORM** : Calcul des tranches tarifaires, des économies T1 et des agrégations.
- **LOAD DIMENSIONS** : Chargement séquentiel des 4 tables de dimensions.

- LOAD FACTS : Chargement par batch (5 000 lignes) des tables de faits avec mécanisme de retry.
- VERIFY : Validation finale par comptage et requêtes de contrôle.

Durée totale du processus ETL : 8 à 12 minutes pour 330 000 lignes.

Outils et technologies utilisées

Cette section présente en détail les outils, bibliothèques et technologies mobilisés pour concevoir, développer et tester le projet Woyofal. Chaque choix a été fait en tenant compte des contraintes de performance, de maintenabilité, d'accessibilité et de pertinence contextuelle.

1. Frontend : React 18 + Vite 5 + Tailwind CSS

L'interface grand public du simulateur a été développée avec React 18, Vite 5 et Tailwind CSS. Cette stack moderne permet une expérience utilisateur fluide avec calculs en temps réel, graphiques Recharts, routing SPA via React Router v6 et gestion des simulations sauvegardées. Vite offre un HMR ultra-rapide en développement et un build optimisé via ESBuild en production.

2. Langage backend : Python 3.10+

Le cœur du pipeline de données a été développé en Python 3.10+, un langage particulièrement adapté aux projets de data engineering. Python présente une grande lisibilité, un écosystème riche de bibliothèques pour la manipulation et l'analyse de données, et une intégration native avec les bases de données et les APIs externes.

3. Base de données : PostgreSQL 14

PostgreSQL 14 Alpine a été choisi comme moteur du Data Warehouse pour ses performances sur les requêtes analytiques, sa robustesse et son support natif des index, des partitions et des vues matérialisées. La connexion est gérée via psycopg2, avec un mécanisme de retry automatique pour la résilience.

4. Génération de données : Faker + NumPy + SciPy

La bibliothèque Faker a été utilisée pour générer des profils utilisateurs réalistes (prénoms sénégalais, zones géographiques). NumPy et SciPy ont permis de modéliser des distributions de consommation statistiquement représentatives, avec variations saisonnières et profils différenciés par zone.

5. Interface analytique : Streamlit

Streamlit a été choisi pour développer rapidement un dashboard analytique connecté directement à PostgreSQL. Il offre un éditeur SQL interactif, des visualisations en temps réel et un support pour les graphiques Plotly et les cartes Folium des 23 zones géographiques du Sénégal.

6. Chatbot IA : OpenRouter API

Le composant AChat.jsx intègre un chatbot flottant basé sur l'API OpenRouter, avec une cascade de six modèles de langage : Llama 3.3 70B (principal), Gemma 3 12B, Qwen 3 8B, Llama 3.1 8B, Mistral 7B et Llama 3.2 3B (ultra-light fallback). Ce mécanisme garantit une réponse même en cas d'indisponibilité d'un modèle. Le chatbot est spécialisé sur les questions tarifaires Senelec et effectue les calculs FCFA/kWh directement dans sa réponse.

7. Déploiement : Vercel + GitHub Actions

L'application React est déployée en production sur Vercel avec CI/CD intégré. Chaque push sur la branche main déclenche automatiquement un build et un déploiement sur le CDN mondial de Vercel. La règle rewrite dans vercel.json garantit le fonctionnement du routage SPA React Router en production.

8. Tableau des bibliothèques complémentaires

Bibliothèque	Rôle dans le projet
pandas	Manipulation des DataFrames CSV et transformations ETL
psycpg2	Client PostgreSQL pour l'ingestion et les requêtes analytiques
Faker	Génération de données utilisateurs réalistes (prénoms sénégalais, zones)
NumPy / SciPy	Calculs statistiques et modélisation des distributions de consommation
Plotly	Graphiques interactifs dans Streamlit et Jupyter Notebooks
Folium	Cartes géographiques des 23 zones de consommation du Sénégal

Bibliothèque	Rôle dans le projet
pytest	Tests unitaires et d'intégration (50+ tests, >85% couverture)
dotenv	Gestion sécurisée des variables d'environnement (clés DB, API)
Docker Compose	Containerisation et déploiement reproductible
Recharts	Graphiques React (courbes, histogrammes) dans le simulateur frontend
Lucide React	Icônes légères et cohérentes dans l'interface utilisateur
axios	Client HTTP pour les appels API en développement local

Interfaces et visualisation

Dans un projet destiné à un large public, l'ergonomie et la lisibilité des interfaces constituent des éléments centraux. L'interface de Woyofal a été pensée pour être intuitive, moderne et accessible, que ce soit pour l'utilisateur final du simulateur ou pour l'analyste de données explorant le dashboard.

1. Application React **Simulateur grand public**

L'application React est accessible à l'adresse frontend-react-wine.vercel.app et offre un parcours utilisateur structuré autour de six composants principaux :

4. `HomePage.jsx` : mini-calculateur interactif FCFA→kWh avec glisseur de montant, affichage temps réel de la tranche atteinte (T1 vert, T2 orange, T3 rouge), FAQ et guide en 3 étapes.
5. `SimulateurRecharge.jsx` : formulaire interactif en mode Direct (FCFA→kWh) ou Inverse (kWh→FCFA), affichage du détail par tranche, cumul avant/après et historique des simulations en `localStorage`.
6. `GuideTarifs.jsx` : explication claire des tranches DPP/PPP avec exemples chiffrés et grille 2026 officielle.
7. `Conseils.jsx` : calculateur de budget par appareil (LED, climatiseur, réfrigérateur, télévision) avec estimation mensuelle de consommation.
8. `AIChat.jsx` : assistant IA flottant basé sur OpenRouter, spécialisé Woyofal, avec rendu Markdown et gestion de l'historique de conversation.
9. `Dashboard.jsx` + `HistoriqueConsommation.jsx` : espace personnel sécurisé (JWT) donnant accès à l'historique des simulations et à la lecture du compteur physique.

2. Moteur de calcul tarifaire **Algorithmes clés**

Mode direct (FCFA → kWh)

L'algorithme applique les déductions (redevance 429 FCFA pour la première recharge du mois, taxe communale 2,5%) puis distribue le montant net progressivement par tranche, en tenant compte du cumul mensuel actuel. Pour chaque tranche, les kWh disponibles sont calculés jusqu'au seuil, puis la tranche suivante prend le relais.

Exemple concret : pour 15 000 FCFA, cumul=0, sans redevance $\rightarrow$ taxe = 375 FCFA, net = 14 625 FCFA $\rightarrow$ T1 : $150 \text{ kWh} \times 82 = 12\,300 \text{ FCFA} \rightarrow$ reste = 2 325 FCFA $\rightarrow$ T2 : $2\,325 / 136,49 = 17,0 \text{ kWh} \rightarrow$ Total = 167,0 kWh en Tranche 2.

Mode inverse (kWh $\rightarrow$ FCFA) Fonctionnalité exclusive

Le calcul inverse est une fonctionnalité absente des outils concurrents identifiés. L'utilisateur saisit le nombre de kWh souhaités et la plateforme calcule le montant exact à recharger. L'algorithme distribue les kWh par tranche en ordre croissant, cumule les coûts, puis applique en sens inverse la taxe communale et la redevance. Le résultat est arrondi au 100 FCFA supérieur.

3. Dashboard Streamlit Espace analytique

Le dashboard Streamlit est destiné aux analystes souhaitant explorer les données du Data Warehouse. Il comprend :

- Connexion dynamique à PostgreSQL via le panneau latéral.
- Requêtes prédéfinies : Top 10 zones par consommation, tendances mensuelles.
- Éditeur SQL personnalisé pour des analyses ad-hoc.
- Visualisations Plotly : courbes de tendance, histogrammes de distribution.
- Carte Folium des 23 zones géographiques du Sénégal avec niveaux de consommation.
- Export des résultats et statistiques du Data Warehouse.

4. Jupyter Notebooks Analyses EDA

Plusieurs notebooks Jupyter ont été développés pour l'exploration exploratoire des données : analyse initiale des distributions et statistiques descriptives, analyse complète incluant corrélations et saisonnalité, et un modèle de prédiction ML des montants de recharge.

Analyse technique du code

Cette section présente la structure du code, les choix d'implémentation et le fonctionnement interne des principaux composants du projet Woyofal. Le développement a été réalisé en séparant clairement les responsabilités entre le frontend React (calcul et interface client) et le backend Python (pipeline de données et API).

1. Structure générale du projet

- frontend-react/src/ : application React avec moteur de calcul JavaScript pur côté client.
- scripts/generate_all_data.py : génération des données synthétiques (CSV).
- scripts/etl/load_to_warehouse.py : pipeline ETL vers PostgreSQL.
- scripts/simulation/recharge_simulator.py : moteur de calcul tarifaire Python.
- api/flask/ : API REST Flask avec routes /api/simulation, /api/ai, /api/consommation, /api/auth.
- dashboards/streamlit/ : dashboard analytique interactif.
- dags/ : DAGs Airflow pour l'automatisation future du pipeline ETL.

2. Classe DataWarehouseLoader (ETL Python)

La classe principale du pipeline ETL gère l'ensemble du processus de chargement avec les méthodes suivantes :

- connect() : connexion PostgreSQL avec mécanisme de retry (3 tentatives).
- load_dim_date() / load_dim_zones() / load_dim_users() : chargement séquentiel des dimensions.
- load_fact_consumption() : ingestion des 300 000 lignes de consommation par batch de 5 000.
- load_fact_recharges() : ingestion des 20 000 transactions de recharges.

Chaque chargement utilise des transactions SQL avec BEGIN/COMMIT et rollback automatique en cas d'erreur. Un système de logs structurés trace chaque étape du pipeline.

3. Classe RechargeSimulator (Python backend)

Le simulateur Python implémente la logique tarifaire de la Décision SENELEC 2025-140 :

- Grille DPP : T1 (0–150 kWh) à 82 FCFA/kWh, T2/T3 (>150 kWh) à 136,49 FCFA/kWh, T4 (>400 kWh) à 162 FCFA/kWh.
- Grille PPP : T1 (0–50 kWh) à 147,43 FCFA/kWh, T2/T3 (>50 kWh) à 189,84 FCFA/kWh.
- Mode Direct : déduction de la taxe communale (2,5%) et de la redevance (429 FCFA), puis calcul progressif par tranche.
- Mode Inverse : calcul à rebours pour déterminer le montant exact nécessaire pour un objectif kWh.

4. Chatbot IA Route /api/ai (Flask)

La route AI implémente un mécanisme de cascade sur 6 modèles OpenRouter avec 2 tentatives par modèle avant de passer au suivant. Le chatbot dispose d'un moteur de calcul local intégré (`_fcfa_to_kwh` et `_kwh_to_fcfa`) permettant de répondre aux questions tarifaires même sans appel API. Un fallback intelligent analyse le message de l'utilisateur et génère une réponse structurée localement si tous les modèles sont indisponibles.

5. Sécurité et bonnes pratiques

- Variables d'environnement : clés API et de connexion stockées dans `.env` géré via `python-dotenv`.
- Transactions SQL : `BEGIN/COMMIT` avec `rollback` automatique en cas d'erreur.
- Rate limiting : `flask-limiter` appliqué sur les routes sensibles de l'API.
- Authentification JWT : tokens côté client pour les fonctionnalités personnalisées.
- Séparation frontend/backend : aucune clé API n'est exposée dans le code React côté client.

6. Points forts de l'implémentation

- Offline-first : moteur de calcul JavaScript pur dans le navigateur, sans dépendance serveur pour la simulation.
- Modularité : chaque classe et fonction a une responsabilité unique et bien définie.

- Performance : batch processing optimisé (5 000 lignes/batch) avec executemany PostgreSQL.
- Portabilité : déploiement via Docker Compose en une seule commande pour le backend.
- Déploiement cloud : frontend en production sur Vercel avec CI/CD GitHub Actions automatique.

Résultats et tests

1. Bilan des fonctionnalités livrées

Fonctionnalité	Description	Statut
Mini-calculateur homepage	Glisseur FCFA, 4 tranches, tranche atteinte en temps réel	☑ Livré
Simulateur direct (FCFA → kWh)	Détail par tranche, cumul mensuel, redevance	☑ Livré
Simulateur inverse (kWh → FCFA)	Fonctionnalité exclusive vs concurrents	☑ Livré
Support DPP / PPP	Les deux types de compteur avec grilles différenciées	☑ Livré
Chatbot IA (AIChat.jsx)	OpenRouter, 6 modèles, fallback local, calculs à la volée	☑ Livré
Historique des simulations	localStorage côté client (backend en développement local)	☑ Livré
Authentification JWT	Connexion / Inscription sécurisées	☑ Livré
Dashboard Streamlit	Connecté à PostgreSQL, carte Folium, éditeur SQL	☑ Livré
Pipeline ETL complet	330 000 lignes, Star Schema, retry automatique	☑ Livré
Déploiement Vercel + CI/CD	Production en ligne, push → déploiement auto	☑ Livré

2. Corrections apportées en cours de développement

Plusieurs bugs significatifs ont été identifiés et résolus au cours du développement :

10. **Bug 1** Page blanche en production (mobile) : le simulateur appelait `http://localhost:5000` en production Vercel. Correction : duplication du moteur de calcul en JavaScript pur côté client.
11. **Bug 2** Variable d'état React obsolète : état React référencé dans le JSX après suppression, provoquant une erreur de compilation. Correction : suppression du bloc JSX orphelin.
12. **Bug 3** Fonction `saveSimulation` manquante : fonction supprimée des imports mais encore appelée dans un handler. Correction : remplacement par sauvegarde `localStorage` directe.
13. **Bug 4** Secret Vercel manquant : `vercel.json` référençait `@api_url_prod` inexistant. Correction : suppression de la clé env.

Défis et leçons apprises

1. Défis rencontrés

Calcul tarifaire progressif et cas limites

La grille Senelec ne s'applique pas sur le montant total mais progressivement : chaque FCFA est d'abord converti aux kWh de la tranche courante jusqu'au seuil, puis la tranche suivante prend le relais. Cet algorithme itératif a nécessité une attention particulière pour les cas limites : cumul déjà en T2, montant insuffisant pour remplir T1, arrondi du calcul inverse, prise en compte simultanée de la redevance et de la taxe communale.

Indépendance du backend en production

Le simulateur appelait initialement une API Node.js locale (localhost:5000). En production sur Vercel, aucun backend n'étant disponible, cela provoquait une page blanche sur mobile. La solution a été de dupliquer le moteur de calcul en JavaScript pur dans le navigateur, adoptant une architecture offline-first. Cette leçon souligne l'importance de concevoir le calcul métier de manière indépendante du serveur dès le départ.

Cohérence visuelle multi-composants

Avec 10+ composants React partageant la même identité visuelle, le risque d'incohérence était élevé. La centralisation des couleurs dans `tailwind.config.js` et dans `index.css` (classes utilitaires custom) a permis de maintenir la cohérence tout au long du développement.

Qualité et représentativité des données synthétiques

En l'absence de données réelles de consommation Woyofal, la génération synthétique a nécessité une modélisation fine des distributions (variations saisonnières, profils par zone, comportements de recharge). Bien que le jeu de données soit réaliste par construction, il présente des limites en termes de représentativité réelle qui devront être adressées lors d'une collecte de données terrain.

2. Leçons apprises

- Architecture offline-first : concevoir le calcul métier côté client dès le départ évite les dépendances runtime coûteuses en production.

- Tests sur mobile réel : le rendu sur desktop ne suffit pas. Les tests sur téléphone ont révélé les bugs de production les plus critiques.
- Séparation des préoccupations : un moteur de calcul pur (sans framework) est plus facile à tester et à réutiliser (CLI, API, frontend).
- Gestion des variables d'état obsolètes : chaque suppression d'état React doit s'accompagner d'une recherche exhaustive des références dans le JSX.
- CI/CD dès le début : l'automatisation du déploiement dès le premier sprint aurait permis de détecter les bugs de production plus tôt.

Conclusion

Le projet Woyofal incarne une alliance réussie entre rigueur technique et utilité sociale, dans un contexte sénégalais marqué par des enjeux énergétiques croissants. En construisant une plateforme de données complète du pipeline ETL au simulateur interactif déployé en production, en passant par un Data Warehouse optimisé, des dashboards analytiques et un chatbot IA ce projet a démontré qu'il est possible de valoriser les données pour un impact concret sur le quotidien des consommateurs.

Ce travail s'inscrit pleinement dans les objectifs de la Sonatel Academy : former des développeurs capables d'identifier des problématiques concrètes, d'y répondre avec des outils modernes, tout en intégrant les contraintes opérationnelles et humaines. Il témoigne de la richesse de l'approche pédagogique basée sur la pratique, les projets et l'impact local.

Sur le plan personnel, cette expérience m'a permis de :

- Consolider mes compétences en Python, SQL, architecture de données et développement d'interfaces modernes (React, Tailwind CSS).
- Approfondir ma compréhension du cycle complet de vie des données, de la génération à la visualisation en passant par l'ingestion ETL.
- Développer une sensibilité aux enjeux opérationnels des projets data dans un contexte africain, notamment la contrainte d'accessibilité mobile et l'importance de la fiabilité tarifaire.
- Maîtriser le déploiement cloud moderne avec CI/CD intégré (Vercel, GitHub Actions).

Au-delà de l'exercice technique, le projet Woyofal m'a offert l'opportunité de donner du sens à mes compétences, en mettant la technologie au service d'une cause essentielle : la transparence tarifaire et l'autonomisation des consommateurs d'énergie.

Perspectives d'évolution

Les perspectives d'évolution à court terme (1–3 mois) incluent le déploiement du backend Flask sur Railway ou Render pour activer l'authentification en production, l'ajout de la tarification PPP dans le mini-calculateur de la page d'accueil, et l'export PDF des simulations.

À moyen terme (3–6 mois), l'intégration d'un pipeline Airflow pour l'automatisation du traitement des données, l'ajout de modèles ML de prédiction de consommation (Prophet/LightGBM), et le développement d'une application mobile React Native constituent des évolutions naturelles.

À long terme, un déploiement cloud complet (AWS ou Azure) avec monitoring en temps réel, l'extension à d'autres secteurs (eau, gaz) et l'adaptation à d'autres pays d'Afrique de l'Ouest permettraient d'élargir significativement l'impact de la plateforme.

En conclusion, ce projet n'est pas une fin, mais un point de départ. Il confirme qu'avec les bons outils, les bonnes méthodes et un cadre d'apprentissage adapté comme celui de la Sonatel Academy, les jeunes développeurs peuvent être les architectes d'une Afrique numérique plus inclusive et plus data-driven.

Bibliographie

Ouvrages imprimés

- McKinney, W. (2022). Python for Data Analysis, 3rd edition. O'Reilly.
Référence principale pour l'utilisation de pandas dans les transformations ETL et l'analyse exploratoire des données.
- Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly.
Ouvrage fondamental pour la conception de pipelines de données robustes et la gestion de la cohérence dans les entrepôts de données.
- Kimball, R. & Ross, M. (2013). The Data Warehouse Toolkit, 3rd edition. Wiley.
Référence pour la méthodologie Star Schema et les meilleures pratiques de modélisation dimensionnelle.
- VanderPlas, J. (2017). Python Data Science Handbook. O'Reilly. Guide complet pour NumPy, Pandas, Matplotlib et Scikit-Learn, utilisés dans les phases d'analyse et de visualisation.
- Hunt, A. & Thomas, D. (2019). The Pragmatic Programmer. Addison-Wesley.
Principes appliqués pour la qualité du code, les tests automatisés et la documentation.

Cours en ligne et tutoriels

- OpenClassrooms. Analysez des données avec Python. <https://openclassrooms.com>
Pour une approche pédagogique des outils d'analyse de données Python.
- Kaggle. Data Engineering & SQL Notebooks. <https://www.kaggle.com/learn>
Source d'inspiration pour le prototypage rapide et la manipulation des données tabulaires.

Webographie

Sites officiels et documentations techniques utilisés pour le développement de la plateforme Woyofal :

- Documentation Python 3.10+. <https://docs.python.org/fr/3/>
- PostgreSQL 14 Documentation. <https://www.postgresql.org/docs/14/>
- React Documentation. <https://react.dev>
- Vite Documentation. <https://vitejs.dev>
- Tailwind CSS Documentation. <https://tailwindcss.com/docs>
- Streamlit Documentation. <https://docs.streamlit.io>
- Plotly Documentation. <https://plotly.com/python>
- Folium Documentation. <https://python-visualization.github.io/folium>
- IPandas Documentation. <https://pandas.pydata.org/docs>
- pytest Documentation. <https://docs.pytest.org>
- Docker Documentation. <https://docs.docker.com>
- OpenRouter API. <https://openrouter.ai/docs>
- Vercel Documentation. <https://vercel.com/docs>
- SENELEC Décision 2025-140 (CRSE). <https://www.senelec.sn>
- Site de la plateforme déployée. <https://frontend-react-wine.vercel.app>
- Dépôt GitHub du projet. <https://github.com/ndoye09/woyofal2026>